SYNAPSE-CRS · LABORATORY OBSERVER-ONLY CONTROL PLANE

Complete User and Technical Guide

Grounded in repository source, API routers, Alembic migrations, frontend routes, and research reports. In-memory hotpatching is **not implemented**. Laboratory envelopes are not /proc telemetry. SAT is not host safety.

API 0.1.0 · Contracts 1.0.0 · Confidentiality: no secrets · University viva pack

Synapse-CRS Complete User and Technical Guide

Classification: Laboratory research software — observer-only
API version: 0.1.0 · **Contracts:** 1.0.0 · **Console:** 0.1.0
Companion files: `IMPLEMENTATION_FACT_SHEET.md`, `DEMO_RUNBOOK.md`, `docs/safety/claims.md`

> **Honesty banner.** The product name mentions “in-memory hotpatching.” **That capability is not implemented.** The system records authorization, stores signed evidence, correlates it, and reasons over it. It does not patch processes, isolate hosts, or execute mitigations. Case state `ACTION` stores a label with `executed: false`.

Part A — Getting started

A1. What is Synapse-CRS?

Synapse-CRS (Autonomous Neuro-Symbolic Cyber Reasoning and In-Memory Hotpatching Engine) is, **in the current tree**, a **fail-closed control plane** plus an **investigation console**:

- FastAPI on PostgreSQL registers **owned lab/development targets**, cases, agents, signed telemetry, correlation findings, hybrid reasoning assessments, and a SHA-256 audit chain.
- A React/Vite console (`frontend/`) visualizes those APIs. It does not add routes or execute host commands.

The long name is a **research aspiration**. Implemented behaviour is listed in `IMPLEMENTATION_FACT_SHEET.md` and `docs/safety/claims.md`.

- Synapse-CRS Complete User and Technical Guide
- Part A — Getting started
  - A1. What is Synapse-CRS?
  - A2. Why was it developed?
  - A3. What problem does it address?
  - A4. Who would use it?
  - A5–A6. Requirements and prerequisites
  - A7–A10. Installation, environment, database, migrations
  - A11–A12. Backend and frontend startup
  - A13. Authentication
  - A14. Verify startup
  - A15. Startup troubleshooting
- Part B — Application user manual
  - B1. /session — Operator lock (SignIn.tsx)
  - B2. / — Command Centre (CommandCentre.tsx)
  - B3. /telemetry — Telemetry operations (TelemetryOps.tsx)
  - B4. /evidence — Evidence Lab (EvidenceLab.tsx)
  - B5. /investigations and /:caseId — Investigation Studio (InvestigationStudio.tsx)
  - B6. /graph — Threat Graph (ThreatGraph.tsx, GraphCanvas.tsx)
  - B7. /reasoning — Reasoning theatre (ReasoningEngine.tsx)
  - B8. /hypotheses — Hypothesis arena (HypothesisArena.tsx)
  - B9. /defence — Defence lab (DefenceLab.tsx)
  - B10. /research and /:phaseId — Research observatory (ResearchObservatory.tsx, data/research.ts)
  - B11. /control — System control (SystemControl.tsx)
- Part C — Technical architecture
- Part D — Backend and codebase
- Part E — API reference
- Part F — Security architecture
- Part G — Operations
- Part H — Limitations and future scope
- Glossary

- References

A2. Why was it developed?

University / research need: a system that can **observe authorized endpoints**, **explain** what it inferred, and **refuse** to act when evidence is incomplete — instead of a SIEM dashboard that silently scores “malicious” or a tool that auto-mitigates.

A3. What problem does it address?

1. Telemetry without enrollment and signatures can be spoofed.
2. Correlation without provenance invents process trees.
3. Neural scores are often presented as probabilities.
4. Formal SAT is often presented as “the host is safe.”
5. Audit logs that the application can UPDATE are not evidence.

Synapse-CRS addresses these **inside a laboratory scope** (`lab` / `development` assets only).

A4. Who would use it?

- A **student/operator** demonstrating the console.
- A **viewer** reading cases/findings.
- An **admin** registering targets, enrolling agents, running correlation/reasoning.
- **Not** a SOC deploying production blocking.

A5–A6. Requirements and prerequisites

- Python 3.12, Node.js 22, PostgreSQL 16, Docker optional.
- `requirements.lock` for Python; `frontend/package.json` for the console.
- Operator-owned lab hosts only. No internet-facing multi-tenant IdP.

A7–A10. Installation, environment, database, migrations

```
cp .env.example .env
# Fill POSTGRES_PASSWORD, SYNAPSE_APP_PASSWORD ([A-Za-z0-9_-]{16,128})
# Fill DATABASE_URL (synapse_app) and MIGRATION_DATABASE_URL (owner) — do not source .env
# SYNAPSE_AUTH_TOKENS = one-line JSON, admin + viewer, unique actors, tokens ≥16 chars
# SYNAPSE_CORS_ORIGINS = http://127.0.0.1:5173,http://localhost:5173  (no *)
```

`.env` is gitignored. Never commit tokens.

Compose (preferred):

```
docker compose --env-file .env -f infra/docker-compose.yml up --build
```

Postgres init (`infra/postgres/init-roles.sh`) creates `synapse_app` **once on an empty volume**. Compose `migrate` runs Alembic as the owner; the API container gets only `DATABASE_URL` and **refuses to start if `MIGRATION_DATABASE_URL` is set**.

Host Postgres:

```
env -u MIGRATION_DATABASE_URL -u DATABASE_URL make migrate   # 0001 … 0006_hybrid_reasoning
```

Do not drop an existing `synapse_test` database used by pytest.

A11–A12. Backend and frontend startup

```
env -u MIGRATION_DATABASE_URL -u DATABASE_URL make api   # 127.0.0.1:8000
cd frontend && npm ci && npm run dev                     # 127.0.0.1:5173
```

`VITE_API_BASE_URL` defaults to `http://127.0.0.1:8000`.

A13. Authentication

There is **no username/password endpoint**. `/session` accepts a bearer token from `SYNAPSE_AUTH_TOKENS`, stores it in `sessionStorage` (`frontend/src/auth/session.ts`), and never renders the full token (`maskToken`).

- **Connect live** — `enterLive`; HUD source `live` / `laboratory` / `empty` / `unavailable`.
- **Demonstration mode** — labelled fixtures in `frontend/src/data/demo.ts`.
- Live failures **must not** load demo rows (`usePlatform.ts`).

A14. Verify startup

```
curl -sS http://127.0.0.1:8000/health   # {"status":"ok","service":"synapse-crs-
api","version":"0.1.0"}
curl -sS http://127.0.0.1:8000/ready    # {"status":"ready","database":"ok",...}
```

Open `/`. HUD: `API live`, `DB ready`. After a live token: **DEMO MODE** gone. If assessments use `dataset_label=synthetic_lab`, HUD **LAB ENVELOPES** (not `/proc`).

A15. Startup troubleshooting

Symptom	Cause	Action
API refuses to start	`MIGRATION_DATABASE_URL` in process env	Unset it; migrate in a separate process
Incomplete config	Missing `DATABASE_URL` or `SYNAPSE_AUTH_TOKENS`	Fill `.env`; do not source it
`/ready` 503	PostgreSQL down or wrong `synapse_app` password	Match `SYNAPSE_APP_PASSWORD`; `pg_isready`
CORS failure	Origin not in `SYNAPSE_CORS_ORIGINS` or `*`	Exact Vite origin
Alembic hit the wrong DB	Inherited `MIGRATION_DATABASE_URL`	`env -u MIGRATION_DATABASE_URL make migrate`

Symptom	Cause	Action
Token rejected	Length < 16 or not in JSON map	HMAC compare in `auth.py`; 401 `authentication_failed`

Part B — Application user manual

Global chrome (`layout/Shell.tsx`): compact **rail** (Command, Telemetry, Evidence, Cases, Graph, Reasoning, Hypotheses, Defence, Research, Control); **HUD** (workspace / target lock / source); lamps API/DB; Session; Refresh; Sign out or Demo; Ctrl/Cmd+K **command palette** (`CommandPalette.tsx`). Skip link “Skip to workspace”.

Data source vocabulary: `demo` | `live` | `empty` | `unavailable` | loading.

B1. `/session` — Operator lock (`SignIn.tsx`)

Purpose. Enter a bearer or demonstration mode.
Sees. Title “Operator lock”; password **Token** (`minLength` 16); **Connect live**; **Demonstration mode**; API unreachable banner if `/health` failed.
Actions. Submit live token → `enterLive` → `/`. Demo → labelled fixtures.
Data. `/health` only (no auth).
Unavailable. Banner “API unreachable … Use demonstration mode.”
Example. Paste admin token, Connect live, confirm HUD not DEMO MODE.

B2. `/` — Command Centre (`CommandCentre.tsx`)

Purpose. Spatial overview, not a three-column generic dashboard.
Sees. Plant meters (control plane, store, enrolled agents, findings); **IntelligenceCore** SVG; **Target lock**; **Latest finding**; **Research honesty** (phases 10–13 scientific stamps); **Activity tape** (audit events + case link); **Journey** path.
Actions. Click target → `/telemetry`; finding → `/evidence?finding=`; case tape → `/investigations/:id`; research ids → `/research/:phaseId`.
Status. Health `ok` / offline; ready `ready` / unknown. Enrollment ≠ telemetry receipt (note on page).
APIs. `/health`, `/ready`, snapshot GETs in `usePlatform` (targets, cases, agents, findings, correlations, risk, assessments, timeline, audit, verify).
Unavailable. Error banner; empty meters; source `unavailable` without demo swap.
Example. Show `live-lab-host`, tape `target.created` / `telemetry.ingested`, LAB ENVELOPES note.

B3. `/telemetry` — Telemetry operations (`TelemetryOps.tsx`)

Purpose. Distinguish **enrollment** from **receipt**.
Sees. Target **channel** cards; agent matrix (status, fingerprint, key generation); **Receipt ticker** from unique `finding.supporting_events` — explicitly **not** a telemetry dump; identity inspector (public key hex, “Public key is not a session token”).
Actions. Click target to lock (`setSelectedTargetId`); click agent; **Open evidence lab**.
APIs. `GET /api/v1/targets`, `GET /api/v1/targets/{id}/agents`, findings nested events. **No GET `/telemetry/events`.**
Example. One enrolled agent, ticker of `network_connection_observed` laboratory envelopes.

B4. `/evidence` — Evidence Lab (`EvidenceLab.tsx`)

Purpose. Forensic workstation: explorer / canvas / inspector / storyline.
Sees. Filter; findings → supporting events; relation map; missing evidence; inspector (epistemic status, engine score, provenance ids); timeline classifications `observed` | `correlated` | `inferred`.
Actions. Select finding/event; `?finding=` deep link; **Threat graph**.
APIs. Findings + `GET /api/v1/reasoning/assessments/{id}/timeline`.
Example. `repeated_network_activity` with six supporting event ids; missing `process_exited` stays missing.

B5. `/investigations` and `/:caseId` — Investigation Studio (`InvestigationStudio.tsx`)

Purpose. Case docket and **linear** state track.
Sees. Filterable dockets; states AUTHORIZED_TARGET → … → SEALED_AUDIT; transitions with `executed=false`; linked findings (snapshot, not a private join table).
Actions. Select docket (URL); admin **Next stored state** POST transition. Disabled/error if demo or viewer. Copy: "ACTION stores a label. It does not change a host."
APIs. `GET/POST /api/v1/cases`, `GET /api/v1/cases/{id}`, `POST .../transitions`.
Example. Case in `OBSERVATION` after one legal step.

B6. `/graph` — Threat Graph (`ThreatGraph.tsx`, `GraphCanvas.tsx`)

Purpose. Full-bleed React Flow canvas.
Sees. Kind filters (target, agent, finding, event); focus search; dagre LR layout; inspector; legend. Edges: `enrolled`, `finding`, `supports`, correlation `rule_id` only (`graphFromSnapshot.ts`).
Actions. Pan/zoom/fit (Controls); select node; **Open in Evidence Lab**. MiniMap hidden under 1280px.
APIs. Same snapshot; no extra graph endpoint.
Example. `live-lab-host` → agent → findings → events.

B7. `/reasoning` — Reasoning theatre (`ReasoningEngine.tsx`)

Purpose. Inspect hybrid stages.
Sees. Vertical stack Telemetry → Features → Neural → Symbolic → Hybrid → Hypothesis → Explanation; engine version + `dataset_label`; **engine score is not a probability**; H4 rationale when selected; limitations; ablation if present.
APIs. `GET .../reasoning/assessments`, timeline. Admin **POST** to create assessments is **not** a console button (would mutate).
Example. Selected `H4_insufficient_evidence` because reconstruction completeness is low; explain "insufficient ≠ malicious."

B8. `/hypotheses` — Hypothesis arena (`HypothesisArena.tsx`)

Purpose. Compare two hypotheses from the **same** assessment payload.
Sees. Left/right selects; status selected/viable/rejected; support score; rationale; supporting ids; contradicting; unresolved/`missing_evidence`.
Example. H4 selected vs H3 rejected.

B9. `/defence` — Defence lab (`DefenceLab.tsx`)

Purpose. Show Z3 result on a **recommendation record**.
Sees. `solver_result` SAT/UNSAT/UNKNOWN; explanation; assumptions; constraint list; violated. Copy: SAT is not host safety; no mitigation executed.
APIs. Assessment `verification` object (from last reasoning run). Additional `POST .../decision-proposals/verify` exists for admin — not a decorative console execute button.
Example. UNSAT with `identity_certain` violated.

B10. `/research` and `/:phaseId` — Research observatory (`ResearchObservatory.tsx`, `data/research.ts`)

Purpose. Copy **repository conclusions**, not a live trainer.
Sees. Phase 10 synthetic_only; 11–13 `not_completed`; limitations; make commands.
Data. Static catalog, not an API.
Example. Phase 13: label JSON inspection ≠ process trees.

B11. `/control` — System control (`SystemControl.tsx`)

Purpose. Health, chain, audit ledger table (seq, type, actor, time, hash).
APIs. `/health`, `/ready`, `GET /api/v1/audit/events`, `GET /api/v1/audit/verify`.
Example. `valid=true`, event_count matching tape.

Part C — Technical architecture

```
Operator browser (Vite :5173)
    Authorization: Bearer <token>
    └── FastAPI (:8000)
            ├── /health  (no DB)
            ├── /ready   (SELECT 1)
            ├── /api/v1/targets|cases|agents|audit|correlation|reasoning
            └── /api/v1/telemetry/ingest  (Ed25519 vs enrolled key)
                    └── PostgreSQL
                          targets, cases, agents, telemetry_events,
                          findings, correlation_*, reasoning_*,
                          audit_events (append-only), audit_chain_state
Packages: contracts, policy, cases, audit, telemetry, agent,
          correlation, reasoning, validation (offline research)
```

Data flow (implemented path).

```
1. Admin POST /api/v1/targets            → eligibility (lab|development)
2. Admin POST /api/v1/targets/{id}/agents → store Ed25519 public key
3. Agent POST /api/v1/telemetry/ingest    → verify signature vs enrolled key
4. Admin POST .../correlation/runs        → findings + risk heuristic
5. Admin POST .../reasoning/assessments   → reconstruct → IF → rules → H1–H4 → Z3
6. Viewer GET findings / assessments / audit/verify
7. Each mutating step appends audit_events in the same SQL transaction
```

There is **no** message bus, **no** WebSocket push, **no** graph database. The threat graph is a client layout of REST snapshot JSON (`graphFromSnapshot.ts`).

Frontend: React 19, TypeScript, Vite, React Router 7, `@xyflow/react` + dagre, sessionStorage session, `apiRequest` timeout 12s, nested error parse.

Backend: FastAPI composition root `synapse_api.main.create_app`; CORS explicit origins; request size guard; SQLAlchemy sessions; Alembic 0001–0006.

Auth: HMAC-SHA256 compare of bearer vs configured tokens (`auth.py`). Roles admin (writes) / viewer (reads). Ingest does not use bearer.

Telemetry: Envelope JSON Schema 1.0.0 → canonical JSON → Ed25519 → persist if agent enrolled and target eligible.

Correlation: Process identity `(agent_id, boot_id, pid, start_ticks)`; max 5000 newest **enrolled** rows.

Reasoning: Reconstruct → features → Isolation Forest unusualness (not P(attack)) → rules → hybrid select → optional counterfactual → Z3 on proposal.

Audit: Same transaction as business write; `compute_event_hash`; verify replays chain.

Part D — Backend and codebase

Path	Role
`apps/api/src/synapse_api/`	HTTP, models, services, Alembic
`packages/contracts`	Pydantic + `API_VERSION`
`packages/policy`	Eligibility
`packages/cases`	Pure transition rules
`packages/audit`	Hash chain
`packages/telemetry`	Normalize, sign, redact
`packages/agent`	procfs collector
`packages/correlation`	Engine + `scoring.py`
`packages/reasoning`	Hybrid + Z3
`packages/validation`	Phase 10–13 fail-closed eval
`frontend/src`	Console
`tests/`	Unit + PostgreSQL integration
`schemas/`	JSON Schema 2020-12
`infra/`	Compose, Postgres init, API image

Request lifecycle. ASGI `_LazyASGIApp` builds FastAPI on first request so imports do not require secrets. Middleware: CORS, body limit. Router dependency `get_principal` or ingest signature. Service function + SQLAlchemy. Errors → `error_body`. Audit append in the write transaction.

Important functions (representative, not every line):

- `parse_auth_tokens` / `authenticate` — `config.py`, `auth.py`
- `evaluate_authorization` — `packages/policy`
- `compute_event_hash` / `verify_chain` — `packages/audit/chain.py`
- `sign_event` / `verify_event` — `packages/telemetry/signatures.py`
- `ingest_telemetry` — `synapse_api.ingest`
- `score_findings` — `synapse_correlation.scoring`
- `assess` / `_select_id` — `synapse_reasoning.hybrid`
- `verify_proposal` — `synapse_reasoning.verification`
- `apiRequest` / `usePlatform` — frontend client + snapshot

Part E — API reference

Error bodies: `{ "error": { "code", "message", "details" } }`. Pagination: `limit` 1–100, `offset` ≥0, `{items,limit,offset,total}`.

Method	Endpoint	Purpose	Auth	Role
GET	`/`	Service summary	none	—
GET	`/health`	Liveness	none	—
GET	`/ready`	DB `SELECT 1`	none	—
POST	`/api/v1/targets`	Register target	bearer	admin
GET	`/api/v1/targets`	List	bearer	any
GET	`/api/v1/targets/{id}`	Get	bearer	any
GET	`/api/v1/targets/{id}/eligibility`	Eligibility	bearer	any
POST	`/api/v1/targets/{id}/revocation`	One-way revoke	bearer	admin
POST	`/api/v1/targets/{id}/agents`	Enroll	bearer	admin
GET	`/api/v1/targets/{id}/agents`	List agents	bearer	any
GET	`/api/v1/agents/{id}`	Get agent	bearer	any
POST	`/api/v1/agents/{id}/revocation`	Revoke agent	bearer	admin
POST	`/api/v1/agents/{id}/rotation`	Rotate key	bearer	admin
POST	`/api/v1/cases`	Open case	bearer	admin
GET	`/api/v1/cases`	List	bearer	any
GET	`/api/v1/cases/{id}`	Detail + transitions	bearer	any
POST	`/api/v1/cases/{id}/transitions`	Next legal state	bearer	admin
GET	`/api/v1/audit/events`	Ledger	bearer	any
GET	`/api/v1/audit/events/{id}`	One event	bearer	any
GET	`/api/v1/audit/verify`	Replay chain	bearer	any
POST	`/api/v1/telemetry/ingest`	Signed envelope	**Ed25519**	enrolled agent
POST	`/api/v1/targets/{id}/correlation/runs`	Run correlation	bearer	admin
GET	`/api/v1/targets/{id}/findings`	Findings	bearer	any
GET	`/api/v1/targets/{id}/correlations`	Links	bearer	any
GET	`/api/v1/targets/{id}/risk`	Risk summary	bearer	any
GET	`/api/v1/findings/{id}`	One finding	bearer	any
POST	`/api/v1/targets/{id}/reasoning/assessments`	Run hybrid	bearer	admin

Method	Endpoint	Purpose	Auth	Role
GET	`/api/v1/targets/{id}/reasoning/assessments`	List	bearer	any
GET	`/api/v1/reasoning/assessments/{id}`	Get	bearer	any
GET	`.../timeline`	Reconstruction	bearer	any
GET	`.../explanation`	Same assessment view	bearer	any
GET	`.../hypotheses`	Same assessment view	bearer	any
POST	`.../decision-proposals/verify`	Extra Z3 proposal	bearer	admin

Typical 4xx: 401 `authentication_required` / `authentication_failed`; 403 `authorization_failed`; 404; 409 eligibility/illegal transition; 422 `invalid_request`. Ready is 503 when `SELECT 1` fails.

Request / response structures (contracts, abbreviated).

Endpoint	Request	Success body (fields)
POST `/api/v1/targets`	`name`, `owner`, `scope.{environment,assets,description}`, timezone-aware `authorization_expiry`, `allowed_capabilities` ⊂ `{telemetry.ingest, evidence.submit, reasoning.request, verification.request}`	`TargetView`: id, name, owner, scope, status, eligibility flags, timestamps
POST `/api/v1/targets/{id}/revocation`	empty	same view, status revoked (one-way)
GET `.../eligibility`	—	`eligible`, reasons
POST `.../agents`	`public_key_hex` (64 hex chars, 32-byte Ed25519)	`AgentView`: id, target_id, status, fingerprint, key_generation, public_key_hex
POST `.../rotation`	new `public_key_hex`	incremented `key_generation`
POST `/api/v1/cases`	`target_id`, `title`	`CaseDetail` + empty `transitions`
POST `.../transitions`	optional comment; engine picks **next legal** state	`CaseDetail`; ACTION always `executed: false`
POST `/api/v1/telemetry/ingest`	`{envelope, signature}` Base64	`TelemetryIngestView` (accepted event id / sequence). **No GET list.**
POST `.../correlation/runs`	optional window	findings persisted; audit event
GET `.../risk`	—	`score` 0–100, `factors`, `finding_count`, `formula` string

Endpoint	Request	Success body (fields)
POST `.../reasoning/assessments`	`window_seconds`, `dataset_label`, `proposal_kind`, `include_counterfactual`	`ReasoningAssessmentView` (hypotheses, neural, verification)
GET `.../timeline`	—	`ReconstructionView` (observations, processes, quality, classifications)
GET `.../explanation` and `.../hypotheses`	—	**same** `ReasoningAssessmentView` as GET assessment (not extra engines)
POST `.../decision-proposals/verify`	`kind`, `authorized`, `in_scope`, `human_approved`	`VerificationView` SAT/UNSAT/UNKNOWN
GET `/api/v1/audit/verify`	—	`valid`, `event_count`, first broken id if any

Ingest envelope must be contract 1.0.0: allowlisted types `process_started`, `process_exited`, `network_connection_observed`; domain `synapse-crs/telemetry-signature/1`. Signature ≠ eligibility; revoked agents fail ingest.

Part F — Security architecture

Implemented: bearer tokens; admin/viewer split; HMAC compare; CORS allowlist; no wildcard; max request bytes; fail-closed settings; target scope lab/dev only; capability allowlist; eligibility before case writes; row lock on transition; enrolled-key ingest; canonical JSON + domain `synapse-crs/telemetry-signature/1`; audit hash chain; append-only triggers for app role; structured errors without exception text; frontend does not treat UI roles as the security boundary.

Not implemented / limitations: OIDC; hardware key protection (`SYNAPSE_AGENT_HOME` file bits are not HSM); DB owner can disable triggers; tokens are shared secrets (ADR 0002); registry does not prove real-world ownership; signature ≠ authorization (enrollment binds key to target); no rate-limit product beyond request size; no mTLS.

Part G — Operations

Error	Cause	What to do
Database unavailable / `/ready` 503	PostgreSQL down, wrong `synapse_app` password, DB not created	`pg_isready`; match `SYNAPSE_APP_PASSWORD`; run init only on empty volume
API unavailable	process not started, port 8000 taken, `create_app` failed settings	`make api` with `MIGRATION_DATABASE_URL` unset; curl `/health`
Invalid token / 401	token <16 chars, not in `SYNAPSE_AUTH_TOKENS`, wrong header	HMAC compare in `auth.py`; `Authorization: Bearer`
Permission denied / 403	viewer on admin route	Use admin actor; UI cannot override
Missing environment variables	fail-closed `Settings`	copy `.env.example`; never `source .env` into the API if it injects migration URL
Migration failures	Alembic pointed at wrong URL; dirty `synapse_test`	`env -u MIGRATION_DATABASE_URL -u DATABASE_URL make migrate`; do not drop pytest DB
Frontend build / tsc	Node version, stale `node_modules`	Node 22; `npm ci`; `npm test`
CORS	origin not listed or `*`	exact Vite origin in `SYNAPSE_CORS_ORIGINS`
Missing telemetry	never ingested; revoked agent; wrong target	POST ingest; console has **no** raw event list
Empty investigations	no cases, or correlation not run	admin POST cases + correlation/runs
Empty graph	snapshot has no entities	enroll + ingest + correlate first
HUD `unavailable`	live token but fetch failed	do **not** switch to demo silently

Part H — Limitations and future scope

`docs/roadmap.md` still describes v0.2–v0.8 as unimplemented; **ignore it as a capability list**. Current tree already contains telemetry through Phase 13 **engineering**. Remaining **scientific** gaps: CIC-IDS2017 not present by default; DMBD trees not downloaded; 12-row synthetic Phase 10 ≠ enterprise detection; Isolation Forest trained on synthetic features; Z3 model ≠ host safety; **no mitigation executor; no hotpatching**. Future work belongs after tests and honest claims, not as a viva invention.

Glossary

Enrolled — admin-registered agent key for one target. **Observed / correlated / inferred** — timeline classifications. **Engine score / support** — not a probability. **SAT** — solver found a model of encoded constraints. **LAB ENVELOPES** — signed synthetic ingest. **DEMO MODE** — browser fixtures.

References

`README.md`, `docs/architecture/*`, `docs/safety/claims.md`, `docs/guides/frontend.md`, `DEMO_RUNBOOK.md`, `apps/api/src/synapse_api/routers/*.py`, `packages/*/src`.

Synapse-CRS documentation pack. Do not treat the product name as a capability list.